University of Kragujevac

Seminar paper from the subject

Software engineering

Website development for the organization of the educational process

Student:

Babaev Bogdan 0866511

Kragujevac, June 2024

Contents

Introduction

In the modern world, information technology plays a key role in various fields of human activity. One of the most sought-after areas is the development of web applications that allow users to access information and perform various tasks. In this course work, we will look at the development of a Python Flask application that provides user authorization on the site.

The relevance of the topic is due to the need to create effective and secure authorization systems to ensure access to various resources and services. The development of such an application will allow you to control access to the materials and functions of the site, as well as increase the security level of the system and learn how to put UML diagrams into practice.

The aim of the work is to develop a Python Flask application that provides authorization for users with various privileges (administrator, teacher, student or guest) based on their username and password. The application should provide different levels of access depending on the user's role, including the ability to change materials, download materials, or just view them. It is also necessary to build various types of diagrams.

Task description

The task is to create a web application that allows users to log in with different access levels (administrator, professor, student, guest) and perform specific functions corresponding to their roles.

After logging in, users should be able to access the functions corresponding to their role.

Develop a website with the ability to login to different accounts and create all kinds of studied diagrams on the subject of Software engineering.

1. Website implementation

The site provides entry for four types of users:

1) The Administrator

2) Professor

3) Guest

4) Student

Each of the users has limited functionality on our site, let's analyze the functionality of each one individually. Let's look at each user individually and their code.

It is worth clarifying that our main program code is written in Python Flask, and user templates are in html.

Flask is a lightweight web framework for the Python programming language that allows you to create web applications quickly and easily. Flask was created by Armin Ronacher and is part of the Pocoo project.

The main features of Flask:

Lightweight: Flask is minimalistic and does not contain a large number of built-in components. This allows developers to select and add only those extensions and libraries that are necessary for a particular project.

Modularity: Flask is built on the basis of a modular architecture, which makes it flexible and extensible.

Ease of use: Flask has a simple and intuitive syntax, which makes it an excellent choice for novice developers and small projects.

Jinja2: Flask uses the powerful Jinja2 template engine, which makes it convenient to work with HTML templates.

WSGI: Flask is based on WSGI (Web Server Gateway Interface), which makes it compatible with most web servers.

Flask version:

The sample code we used does not specify a specific version of Flask. However, to ensure that all the code examples work correctly, it is recommended to use the latest stable version of Flask.

At the time of writing the answer, one of the latest versions is Flask 2.0.1.1.

1. Administrator functionality:

Appointment of professors: The administrator can appoint other users as professors. To do this, he must enter the username and submit the form. If the user exists, his role will be changed to "professor".

Viewing the list of users: The administrator can see a list of all users and their current roles.

The administrator's code:

```
<!DOCTYPE html>
<html>
<head>
    <title>Admin Page</title>
</head>
<body>
    <h2>Welcome, Admin!</h2>
    <h3>Assign a user as a professor:</h3>
    <form method="post">
        <label for="username">Username:</label>
        <input type="text" id="username" name="username"><br><br>
        <input type="submit" value="Appoint a professor">
    </form>
    <h3>List of users:</h3>
    <ul>
        {% for username, user in users.items() %}
            <li>{{ username }} - {{ user.role }}</li>
        {% endfor %}
    </ul>
    <a href="{{ url_for('logout') }}">Logout</a>
</body>
</html>
```

2. The Professor functional:

Uploading material: The professor can upload educational materials by entering text into the form and submitting it.

Editing the material: The professor can edit existing materials by specifying the index of the material and the new text.

Viewing materials: The professor can see a list of all uploaded materials.

```
<!DOCTYPE html>
<html>
<head>
    <title>Professor Page</title>
</head>
<body>
    <h2>Welcome, Professor!</h2>
    <h3>Upload the material:</h3>
    <form method="post">
        <input type="hidden" name="action" value="upload">
        <textarea name="material" rows="4" cols="50"></textarea><br><br>
        <input type="submit" value="Загрузить">
    </form>

    <h3>Materials:</h3>
    <ul>
        {% for index, material in materials %}
            <li>{{ material }}</li>
        {% endfor %}
```

```
    </ul>

    <h3>Edit the material:</h3>
    <form method="post">
        <input type="hidden" name="action" value="edit">
        <label for="index">The index of the material:</label>
        <input type="number" id="index" name="index"><br><br>
        <textarea name="new_text" rows="4" cols="50"></textarea><br><br>
        <input type="submit" value="Редактировать">
    </form>

    <a href="{{ url_for('logout') }}">Logout</a>
</body>
</html>
```

3. Student functional:

Viewing materials: The student can see a list of all uploaded materials.

```
<!DOCTYPE html>
<html>
<head>
    <title>Student Page</title>
</head>
<body>
    <h2>Welcome, Student!</h2>
    <h3>Materials:</h3>
    <ul>
        {% for material in materials %}
            <li>{{ material }}</li>
        {% endfor %}
    </ul>
    <a href="{{ url_for('logout') }}">Logout</a>
</body>
</html>
```

4. The guest functional:

Viewing a welcome image: The guest sees a page with a welcome image.

```
<!DOCTYPE html>
<html>
<head>
    <title>Guest Page</title>
</head>
<body>
    <h2>Welcome to the university's website</h2>
    </h2> while there is still no other information here -</h2>
    </h2>   pay attention to the view of the city of Leskovac!</h2>
    <img src="{{ url_for('static', filename='welcome.jpg') }}" alt="Welcome">
    <a href="{{ url_for('logout') }}">Logout</a>
</body>
</html>
```

Thus, the application provides different functionality for four types of users with different access levels and capabilities depending on their role in the system.

1.2 Website interface

Let's take a look at the functional part of the website code, which was written in Python, as mentioned earlier. The code was written using the Flask framework. The project has the following structure, as shown in Fig 1:

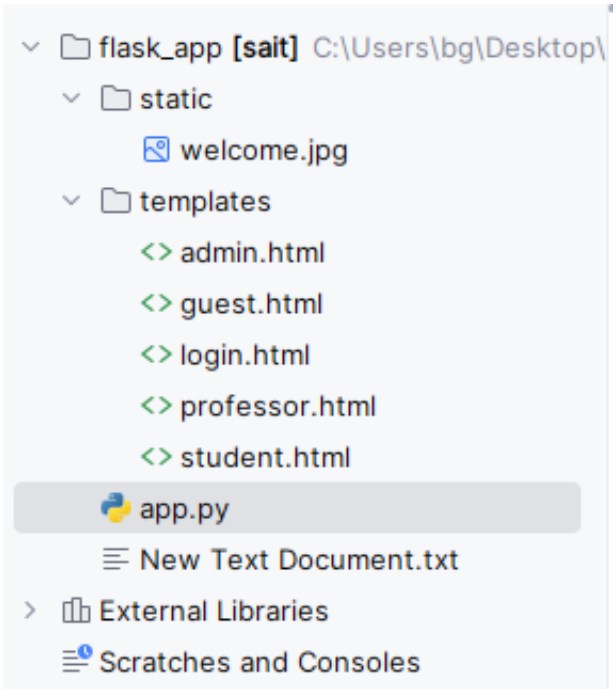

Fig 1. Project structure

We have one main file app.py which stores all the necessary functions that link the rest of the files and make the site workable.

The templates folder contains 5 html files that are responsible for displaying the interface of pages on the site. As an example, we will show the page of the professor and the admin in fig 2.

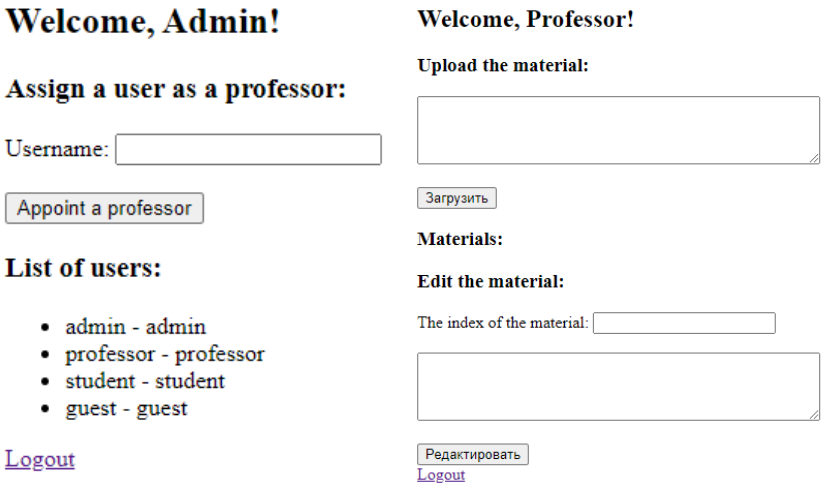

Fig 2. Website interface

1.3 Site functionality

The site code is built in such a way that user data is first determined, after the routes to the home page and the login route, this is approximately a description of the actions for each user and the logic of correct and incorrect user name and password entry.

```
app = Flask(__name__)
app.secret_key = 'your_secret_key'

users = {
    "admin": {"password": "adminpass", "role": "admin"},
    "professor": {"password": "professorpass", "role": "professor"},
    "student": {"password": "studentpass", "role": "student"},
    "guest": {"password": "guestpass", "role": "guest"}
}

materials = []

@app.route('/')
def home():
    return redirect(url_for('login'))

@app.route( rule: '/login', methods=['GET', 'POST'])
def login():
    if request.method == 'POST':
        username = request.form['username']
        password = request.form['password']
        user = users.get(username)
        if user and user['password'] == password:
            session['username'] = username
            session['role'] = user['role']
            return redirect(url_for(user['role']))
        else:
            return "Incorrect username or password"
    return render_template('login.html')

@app.route( rule: '/admin', methods=['GET', 'POST'])
def admin():
    if session.get('role') != 'admin':
        return "Access is denied"
```

Fig 3. Site functionality

After that, the routes of user interactions with the site, opening pages and transferring information from one user to another are scheduled, after that there is a route to log out and launch the application.

```
@app.route('/guest')
def guest():
    if session.get('role') == 'guest':
        return render_template('guest.html')
    return "Access is denied"

@app.route('/logout')
def logout():
    session.pop('username', None)
    session.pop('role', None)
    return redirect(url_for('login'))

if __name__ == '__main__':
    app.run(debug=True)
```

Fig 4. The route of the guest, logout and launch of the application

2. UML Diagram

2.1 Use case

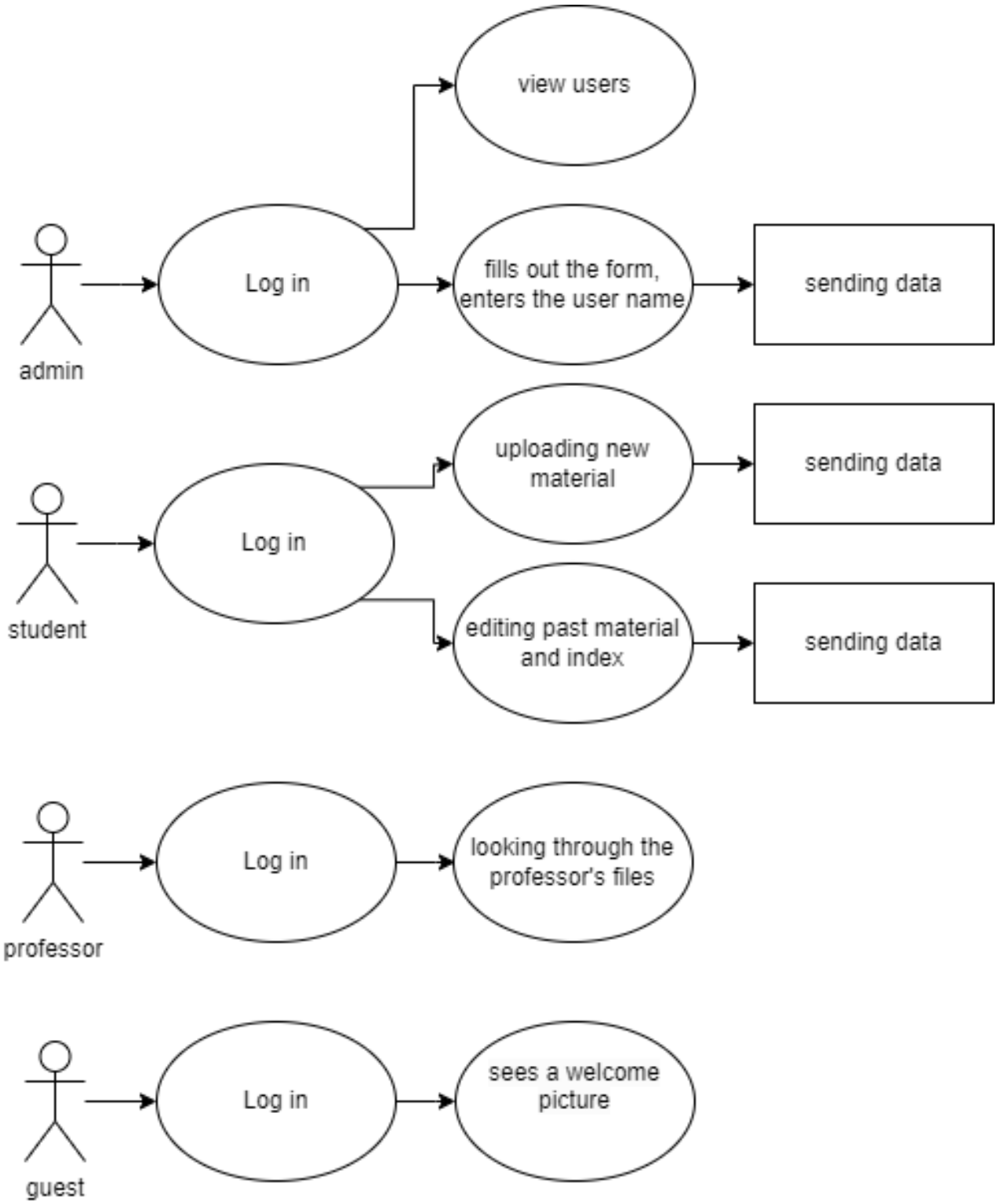

Fig 5. Use case diagram

2.2 Activity Diagram

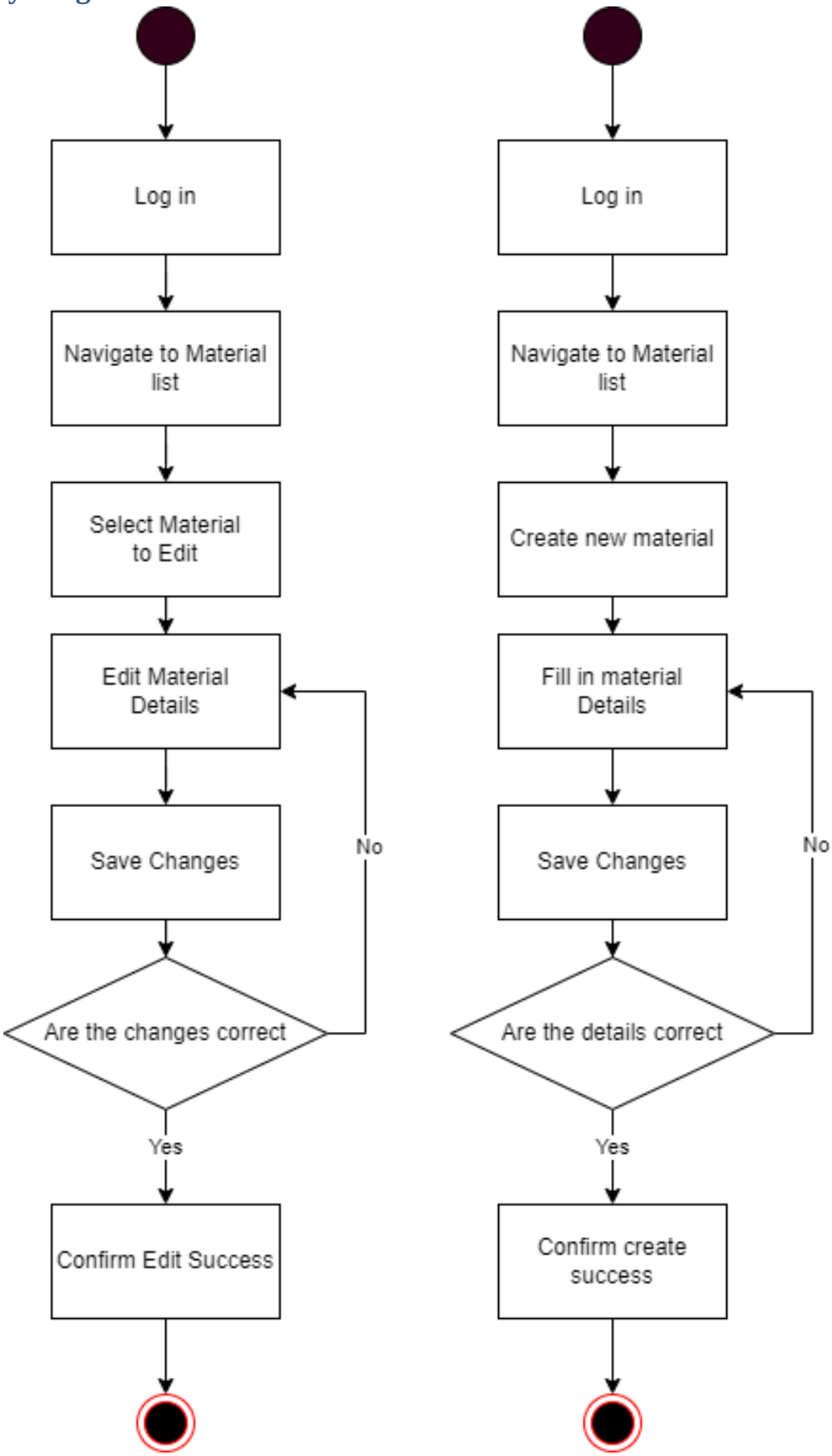

Fig 6. Activity Diagram

2.3 Sequence Diagrams

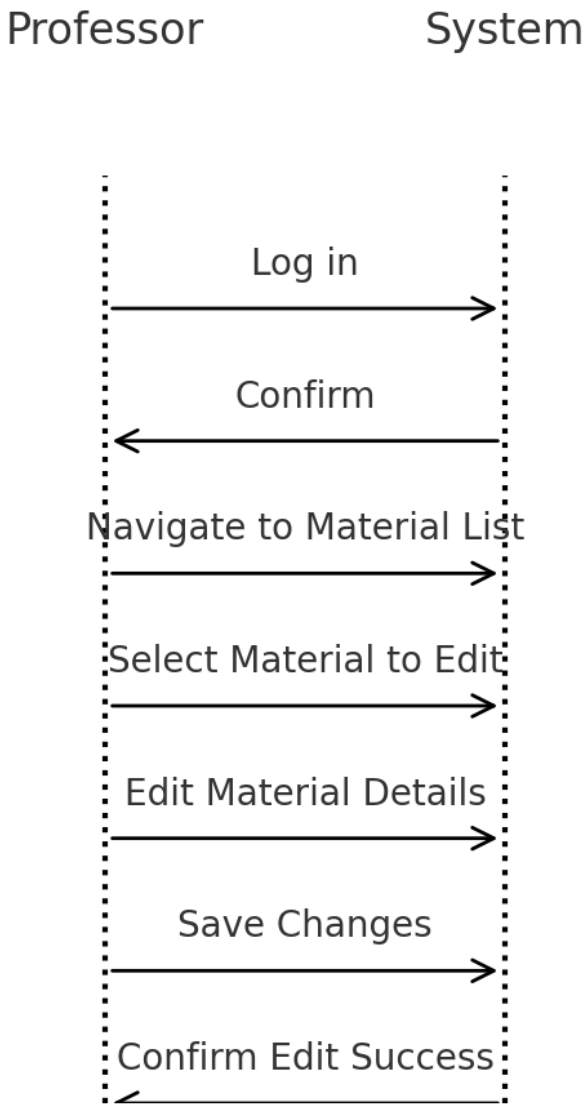

Fig 7. Sequence Diagram

Conclusion

In conclusion, this project successfully demonstrates the development of a Python Flask web application for user authorization, coupled with the implementation of UML diagrams. By leveraging Flask, the application offers a lightweight yet powerful framework for building web applications with different user roles and functionalities. The use of UML diagrams, including use case, activity, and sequence diagrams, provides a comprehensive understanding of the system's behavior and interactions. Through this project, valuable insights have been gained into the practical application of software engineering principles in web development, particularly in terms of user authentication and system design.

This work lays a solid foundation for further exploration and enhancement of web application development using Python Flask, as well as the utilization of UML diagrams for system analysis and design. With the continuous evolution of technology and the increasing demand for secure and user-friendly web solutions, the knowledge and skills gained from this project are invaluable for future endeavors in the field of software engineering and web development.

List of References

Books

1. Grinberg, M. (2018). *Flask Web Development: Developing Web Applications with Python*. O'Reilly Media.
2. Ronacher, A. (2018). *Explore Flask*. The Pallets Projects.
3. Ramalho, L. (2015). *Fluent Python: Clear, Concise, and Effective Programming*. O'Reilly Media.
4. Beazley, D. M., & Jones, B. K. (2013). *Python Cookbook: Recipes for Mastering Python 3*. O'Reilly Media.

Articles and online resources

1. Pallets Projects. *Flask Documentation*. Retrieved from https://flask.palletsprojects.com/en/2.0.x/
2. Real Python. *Flask by Example: Part 1*. Retrieved from https://realpython.com/flask-by-example-part-1-project-setup/
3. DigitalOcean. *How To Build a Web Application Using Flask and Deploy It to the Cloud*. Retrieved from https://www.digitalocean.com/community/tutorials/how-to-build-a-web-application-using-flask-and-deploy-it-to-the-cloud
4. Hackers and Slackers. *Flask 101: Getting Started*. Retrieved from https://hackersandslackers.com/flask-101/

Additional resources

1. Stack Overflow. *Common Flask Questions and Answers*. Retrieved from https://stackoverflow.com/questions/tagged/flask
2. GitHub. *Flask Example Projects*. Retrieved from https://github.com/pallets/flask/tree/main/examples
3. Python Software Foundation. *Python 3 Documentation*. Retrieved from https://docs.python.org/3/
4. Coursera. *Web Applications with Python and Flask by University of Michigan*. https://www.coursera.org/learn/python-flask